

252-0027

Einführung in die Programmierung Übungen

Woche 3: Einführung Java

Fabian Rohlik

Departement Informatik

ETH Zürich

Organisatorisches

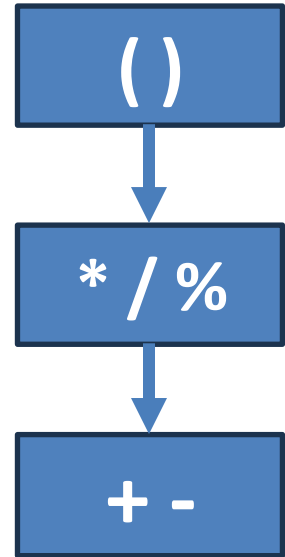
- Mein Name: Fabian Rohlik
- Bei Fragen: frohlik@student.ethz.ch
- Notizen: cs.rohlik.net, dann auf EProg Übungsstunde klicken
- Neue Aufgaben: **Dienstag Abend** (im Normalfall)
- Abgabe der Übungen bis **Dienstag Abend (23:59)** Folgewoche
 - Ihr könnt so oft commit & push machen wie ihr wollt
 - diese Woche gibt es noch keinen Bonus
 - macht Moodle Quizze (um Gefühl dafür zu bekommen, Prüfung...)

Hi, Java

(kurze Syntax-Einführung)

Auswertung – Ordnung

- Operand wird vom Operator mit höherer Rangordnung («precedence») verwendet
- Wenn zwei Operatoren dieselbe Rangordnung haben, dann entscheidet die Assoziativität
- Wenn etwas anderes gewünscht wird: Klammern verwenden!



Arithmetische Operatoren nach Precedence



Operator Type	Operators
Postfix	expr++ , expr--
Unary	++expr , --expr
Multiplicative	*, / , %
Additive	+, -
Relational	< , > , <= , >= , instanceof
Equality	== , !=
Logical AND	&&
Logical OR	
Ternary	? :
Assignment	= , += , -=

Assoziativität

- Die **Assoziativität** («**associativity**») eines Operators bestimmt, wie ein Operand verwendet wird
- Ist ein Operator $\otimes$
 - **Linksassoziativ** («**left-associative**») $\rightarrow X \otimes Y \otimes Z$ gleich $(X \otimes Y) \otimes Z$
 - **Rechtsassoziativ** («**right-associative**») $\rightarrow X \otimes Y \otimes Z$ gleich $X \otimes (Y \otimes Z)$

Arithmetische Operatoren nach Precedence



Operator Type	Operators
Postfix	expr++ , expr--
Unary	++expr , --expr
Multiplicative	*, / , %
Additive	+, -
Relational	< , > , <= , >= , instanceof
Equality	== , !=
Logical AND	&&
Logical OR	
Ternary	? :
Assignment	= , += , -=

Arithmetische Operatoren nach Precedence



Operator Type	Operators	Associativity
Postfix	expr++ , expr--	Left to Right
Unary	++expr , --expr	Right to Left
Multiplicative	* , / , %	Left to Right
Additive	+ , -	Left to Right
Relational	< , > , <= , >= , instanceof	Left to Right
Equality	== , !=	Left to Right
Logical AND	&&	Left to Right
Logical OR		Left to Right
Ternary	? :	Right to Left
Assignment	= , += , -=	Right to Left

Arithmetische Operatoren nach Precedence

-> IntelliJ demo



Operator Type	Operators	Associativity
Postfix	expr++ , expr--	Left to Right
Unary	++expr , --expr	Right to Left
Multiplicative	* , / , %	Left to Right
Additive	+ , -	Left to Right
Relational	< , > , <= , >= , instanceof	Left to Right
Equality	== , !=	Left to Right
Logical AND	&&	Left to Right
Logical OR		Left to Right
Ternary	? :	Right to Left
Assignment	= , += , -=	Right to Left

**Bei Ausdrücken, die verschiedene Typen
enthalten, „gewinnt“ der genauere Typ.**

-> Beispiel an der Tafel

Auswertung – Beispiele

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println(2/3+4.5/3);  
4     }  
5 }
```

Output: 1.5

Auswertung – Beispiele

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println(8%3+2.5+4/3);  
4     }  
5 }
```

Output: 5.5

Auswertung – Casting

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println((double)5/2);  
4     }  
5 }
```

Output: 2.5

Auswertung – Casting

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println((double)5/2);  
4     }  
5 }
```

Output: 2.5

the type casting operator has a higher precedence than the division operator, so the 5 is casted, then 5.0 is divided by 2

Implizites Casting

-> IntelliJ demo

Auswertung - Typ

- Gleiche Operanden = Klarer Typ

int + int = int

double + double = double

- Was passiert aber, wenn:

double + int = ?

Implizites Casten

$$\text{double} + \text{int} = \text{double}$$

$$\text{String} + \text{int} = \text{String}$$

Bei Ausdrücken, die verschiedene Typen enthalten, „gewinnt“ der genauere Typ.

Auswertung – String

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println("Wir schreiben das Jahr "+2023/2*2.0);  
4     }  
5 }  
6
```

Output: Wir schreiben das Jahr 2022.0

Aufgabe 1

```
1  class MyClass{  
2      public static void main(String[] args){  
3          System.out.println(9 / 3 * 2 + 1 + "" + 2 * 3 + (4 * 3) % 3);  
4      }  
5  }
```

760

Aufgabe 1

```
1  class MyClass{  
2      public static void main(String[] args){  
3          System.out.println(9 / 3 * 2 + 1 + "" + 2 * 3 + (4 * 3) % 3);  
4      }  
5  }
```



760

Aufgabe 1

```
1  class MyClass{  
2      public static void main(String[] args){  
3          System.out.println(9 / 3 * 2 + 1 + "" + 2 * 3 + (4 * 3) % 3);  
4      }  
5  }
```

↓
760

Aufgabe 1

```
1  class MyClass{  
2      public static void main(String[] args){  
3          System.out.println(9 / 3 * 2 + 1 + "" + 2 * 3 + (4 * 3) % 3);  
4      }  
5  }
```

6 0

↓
760

Aufgabe 1

```
1 class MyClass{
2     public static void main(String[] args){
3         System.out.println(9 / 3 * 2 + 1 + "" + 2 * 3 + (4 * 3) % 3);
4     }
5 }
```

760

Auswertung – Prüfungsbeispiel

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println(12/4*3+("X"+7%3)+18%5*2);  
4     }  
5 }
```

Output: 9X16

Auswertung – Prüfungsbeispiel

()

* / %

+ -

```
1
2 public class Main {
3     public static void main(String[] args) {
4         System.out.println("Wir schreiben das Jahr " + 2000 + 24);
5     }
6 }
```

Output: Wir schreiben das Jahr 200024

Aufgabe 2

```
1  class MyClass{  
2      public static void main(String[] args){  
3          System.out.println(8 / 5 + 0.5 + 5 / 2 + (8 % 3) * 1.0);  
4      }  
5  }
```

5.5

Kurzschlussauswertung («short-circuit evaluation»)

Bei Berechnung von `p && q` und `p || q`

- Java wertet zuerst den linken Operanden (`p`) und dann den rechten (`q`) aus
- die Auswertung wird beendet, sobald das Ergebnis feststeht.

Kurzschlussauswertung («short-circuit evaluation»)

Bei Berechnung von `p && q` und `p || q`

- Java wertet zuerst den linken Operanden (`p`) und dann den rechten (`q`) aus
- die Auswertung wird beendet, sobald das Ergebnis feststeht.
 - Bei `p && q`: Falls `p == false`, dann ist `p && q == false`
 - Bei `p || q`: Falls `p == true`, dann ist `p || q == true`

p	q	p && q
true	true	true
true	false	false
false	true	false
false	false	false

p	q	p q
true	true	true
true	false	true
false	true	true
false	false	false

In **diesen Fällen** ist keine Auswertung von `q` nötig!

Kahoot

Escaping-Beispiele

(nur um ein Gefühl dafür zu bekommen)

-> IntelliJ demo

Escape-Sequenz	Name	Ausgabe / Effekt
\"	Doppeltes Anführungszeichen	Fügt " in den String ein.
\'	Einfaches Anführungszeichen	Fügt ' in den String ein.
\\	Backslash	Fügt \ in den String ein.
\n	Newline (Zeilenumbruch)	Springt in die nächste Zeile.
\t	Tabulator	Fügt einen horizontalen Tab-Abstand ein.
\r	Carriage Return (Wagenrücklauf)	Springt an den Anfang der aktuellen Zeile.
\b	Backspace (Rücktaste)	Löscht das direkt davor stehende Zeichen.
\f	Form Feed (Seitenvorschub)	Erzwingt einen Seitenumbruch (wird in modernen Konsolen kaum noch verwendet).
\uXXXX	Unicode	Fügt ein spezifisches Zeichen anhand seines 4-stelligen Hexadezimal-Codes ein (z. B. \u00A9 für das Copyright-Zeichen).

Vorbesprechung Übung 3

Aufgabe 1: Fehlerbehebung

In diesem Projekt befindet sich das fehlerhafte Programm `FollerVehler.java` im separaten Ordner `broken`. Dieser ist im Projekt als *Sources* markiert, aber zusätzlich *Excluded*, damit fehlerhafte Dateien beim Start das Projekt nicht blockieren. Finden und beheben Sie alle Fehler im Programm. IntelliJ zeigt an, wo Fehler auftreten, doch Sie müssen selbst herausfinden, was das Problem ist und wie es behoben werden kann. Nach erfolgreicher Korrektur sollte das Programm folgendes ausgeben:

```
Hello world
```

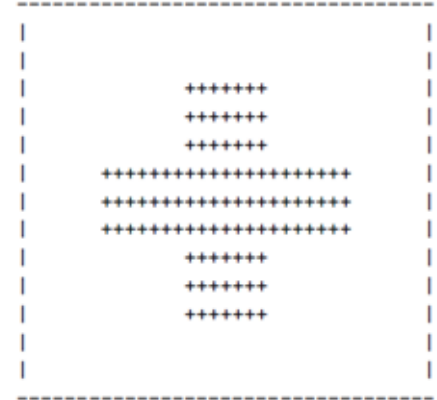
```
Gefällt Ihnen dieses Programm?
```

```
Ich habe es selbst geschrieben.
```

Aufgabe 2: Schweizerfahne

Statt eine Vorlage zu benützen, schreiben Sie in dieser Aufgabe ein Programm von Grund auf selbst. IntelliJ wird Ihnen allerdings etwas Schreibarbeit abnehmen.

1. Erstellen Sie eine neue Java-Datei "SwissFlag.java". Rechts-Klicken Sie dafür den *src*-Ordner und wählen Sie *New* → *Java Class*. Geben Sie den Namen der Klasse ein ("SwissFlag.java") und bestätigen Sie mit der Enter-Taste. Danach wählen Sie *Add*, wenn Sie gefragt werden, ob Sie die Datei zu git hinzufügen möchten.
2. Erstellen Sie zuerst eine leere *main*-Methode, so wie Sie es bei anderen Programmen gesehen haben.
3. Erweitern Sie das Programm so, dass es die Schweizerfahne in der Konsole ausgibt. Die Fahne könnte ungefähr wie folgt aussehen, Sie dürfen aber auch eine grössere oder schönere Version entwerfen:



Teilen Sie das Programm in mehrere Methoden auf, welche von der *main*-Methode aufgerufen werden. Damit sorgen Sie dafür, dass weniger Wiederholungen von Code-Stücken vorkommen, was das Ändern des Programms deutlich einfacher macht.

Aufgabe 3: Eingabe und Zufall

Aufgabe 3: Eingabe und Zufall

In dieser Aufgabe arbeiten Sie mit der Eingabe und Ausgabe von Java und lernen die Klassen `Scanner` und `Random` näher kennen.

1. Schreiben Sie ein Programm "Adder.java", welches zwei ganze Zahlen einliest und die Summe davon ausgibt. Sie sollen dafür die `Scanner`-Klasse verwenden, wie in der Vorlesung gezeigt. Das Programm soll nach der ersten Zahl fragen:

Geben Sie Zahl 1 ein:

dann nach der zweiten Zahl:

Geben Sie Zahl 2 ein:

und schliesslich, wenn Sie zum Beispiel "4" und "1999" eingeben, folgendes ausgeben:

4 + 1999 = 2003

Sie können davon ausgehen, dass nur ganze Zahlen eingegeben werden. Testen Sie das Programm mit verschiedenen Zahlen.

2. Schreiben Sie ein Programm `Wuerfel.java`, das einen Würfel simuliert. Hierbei soll eine positive ganze Zahl `N` eingelesen werden, welche die Anzahl der Seiten des Würfels repräsentiert. Der übliche Würfel hat 6 Seiten, jedoch existieren auch Würfel mit 12, 16, 20 (siehe Abbildung 1) und mehr Seiten. Jede Seite trägt eine unterschiedliche Zahl, die von 1 bis `N` (inklusive `N`) reicht.

Das Programm soll den Wurf simulieren, indem es die Klasse `Random` verwendet. Ein möglicher Ablauf des Programmes könnte folgendermassen aussehen:

Wie viele Seiten hat Ihr Würfel?

Der Benutzer gibt eine Zahl ein, z.B. 20, danach wird gewürfelt:

Es wurde eine 17 gewürfelt!



3. Schreiben Sie ein Programm `ChatGTP.java`, welches die Interaktion mit einem AI-Chatbot simuliert. Das Programm soll den Benutzer begrüßen, Sie nach Ihrem Namen und Alter fragen, und dem Benutzer Ihre Glückszahl verraten. Die Glückszahl soll zufällig gewählt werden, in dem Sie die `Random` Klasse benutzen. Ein möglicher Ablauf des Programmes könnte folgendermassen aussehen:

Guten Tag! Ich bin ChatGTP, der beste Chatbot, denn es gibt! Wie heissen Sie?

Ein Namen wird eingegeben, z.B. Alice, danach antwortet der Chatbot auf diesen Input:

Sehr erfreut Alice! Wie alt sind Sie?

Ein Alter wird eingegeben, z.B. 23, danach antwortet der Chatbot auf diesen Input:

Mittels dieser Informationen habe ich Ihre Glückszahl gefunden! Die Glückszahl lautet 42.

Aufgabe 4: Berechnung

2. Vervollständigen Sie "SharedDigit.java". In der Main-Methode sind zwei int Variablen a und b deklariert und mit einem Wert zwischen 10 und 99 (einschliesslich) initialisiert. Das Programm soll einer int Variablen r einen bestimmten Wert zuweisen. Wenn a und b eine Ziffer gemeinsam haben, dann wird r die gemeinsame Ziffer zugewiesen (wenn a und b beide Ziffern gemeinsam haben, dann kann eine beliebige Ziffer zugewiesen werden). Wenn es keine gemeinsame Ziffer gibt, dann soll -1 zugewiesen. Sie brauchen für dieses Programm keine Schleife.

Beispiele:

- Wenn a: 34 und b: 53, dann ist r: 3
- Wenn a: 10 und b: 22, dann ist r: -1
- Wenn a: 66 und b: 66, dann ist r: 6
- Wenn a: 34 und b: 34, dann ist r: 3 oder 4

Testen Sie Ihre Loesung mit a gleich 34 und b gleich 43. Was liefert Ihr Programm?

2. Vervollständigen Sie "SumPattern.java". In der Main-Methode sind drei int Variablen a, b, und c deklariert und mit irgendwelchen Werten initialisiert. Wenn die Summe von zwei der Variablen die dritte ergibt, nehmen wir an dass $a + c == b$, so soll die Methode "Moeglich. $a + c == b$ " ausgeben (wobei die Werte für a, b, und c einzusetzen sind). Wenn das nicht der Fall ist, dann soll die Methode "Unmoeglich." ausgeben.

Beispiele:

- Wenn a: 4, b: 10, c: 6, dann wird "Moeglich. $4 + 6 == 10$ " oder "Moeglich. $6 + 4 == 10$ " ausgegeben.
- Wenn a: 2, b: 12, c: 0, dann wird "Unmoeglich." ausgegeben.

3. Vervollständigen Sie "AbsoluteMax.java". In der Main-Methode sind drei int Variablen a, b, und c deklariert und mit irgendwelchen Werten initialisiert. Das Programm soll einer int Variable r den grössten absoluten Wert von a, b, und c zuweisen.

Aufgabe 5: EBNF (macht Moodle-Quiz)

1. Erstellen Sie eine Beschreibung $\langle \text{pyramid} \rangle$, welche als legale Symbole genau jene Wörter zulässt, welche aus einer Folge von strikt aufsteigenden, gefolgt von einer Folge von strikt absteigenden Ziffern bestehen. Beispiele sind: 14, 121, 1221, 1341.
Sie dürfen annehmen, dass das leere Wort auch zugelassen wird. (Als Challenge können Sie probieren, das leere Wort auszuschliessen.)
2. Erstellen Sie eine Beschreibung $\langle \text{digitsum} \rangle$, welche als legale Symbole genau jene natürlichen Zahlen zulässt, deren Quersumme eine gerade Zahl ist.
3. Erstellen Sie eine Beschreibung $\langle \text{xyz} \rangle$, welche genau alle Wörter zulässt, die aus X, Y und Z bestehen und bei welchen jedes X mindestens ein Y im Teilwort links und rechts von sich hat. Beispiele sind: Z, YXY, YXXY, ZYXYY.
4. Erstellen Sie eine Beschreibung $\langle \text{term} \rangle$, welche als legale Symbole genau alle wohlgeformten arithmetischen Terme, bestehend aus positiven ganzen Zahlen, Variablen (x, y, z), Addition und Klammern zulässt. Geklammerte Terme müssen mindestens eine Addition enthalten. Beispiele sind: $1 + 4$, $(1 + 4)$, $1 + (3 + 4)$, $(1 + 1) + x + 5$.

Nachbesprechung Übung 2

Lösung Übung 2, Aufgabe 4

1. Erstellen Sie eine Beschreibung `<geradezahl>`, die als legale Symbole alle geraden Zahlen (d.h. Zahlen, die ohne Rest durch 2 teilbar sind) zulässt. Beispiele sind +02, 4, 10, -20.

-> in neuem Fenster durchgehen

Lösung Übung 2, Aufgabe 4

3. Erstellen Sie eine Beschreibung `<x2ygemischt>`, die als legale Symbole genau jene Wörter zulässt, in denen für jedes "X" zwei "Y" als Paar auftreten. Beispiele sind `XYX`, `YYX`, `XYYYXX`, `XXYYYY`. Die Beschreibung schließt den leeren String nicht aus.

-> in neuem Fenster durchgehen

Lösung Übung 2, Aufgabe 4

3. Erstellen Sie eine Beschreibung `<x2ygemischt>`, die als legale Symbole genau jene Wörter zulässt, in denen für jedes "X" zwei "Y" als Paar auftreten. Beispiele sind `XYX`, `YYX`, `XYYYXX`, `XXYYYY`. Die Beschreibung schliesst den leeren String nicht aus.

```
<x2ygemischt> <- [ "X" <x2ygemischt> "YY" <x2ygemischt>  
| "YY" <x2ygemischt> "X" <x2ygemischt> ]
```

Lösung Übung 2, Aufgabe 4

5. Erstellen Sie eine EBNF Beschreibung `<doppelt>`, die als legale Symbole genau jene Wörter zulässt, in denen die doppelte Anzahl "Y" nach einer Folge von "X" auftritt. Beispiele sind `XYX`, `XXYYYYX`, usw. Die Beschreibung schliesst den leeren String nicht aus.

Danke!